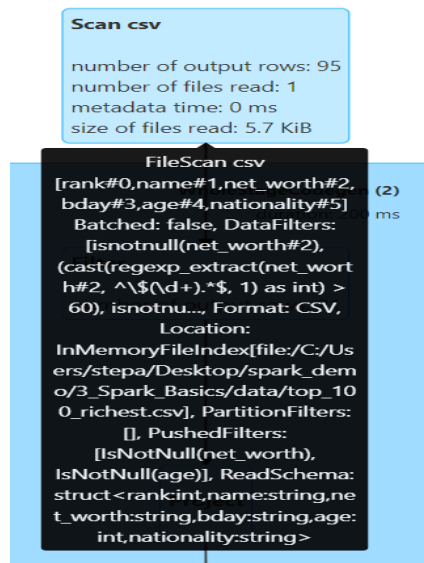


So, since you're here, it means you're really here to check out some cool stuff, so I won't keep you waiting.

Let's start with the simplest part that's already in the code. I think you remember the line

```
df_last = df_prelast.filter(col('age').isNotNull()).select(col('rank'), col('name'), col('net_worth'),  
col('bday'), col('age'), col('nationality')).
```

It was almost at the very end. And here we see a filter, specifically to ensure that the col('age') field isn't null. But what if we take a look at the execution plan?



I think you can see that we have **PushedFilters**, which contains two non-null checks, and one of them is precisely for the **age** field. This means that Spark pushed the filtering for this field directly into the file-reading stage. You can see that the number of output rows is 95 - spoiler alert: there are 100 total rows in the file. This means that Spark actually performed this filtering, even though we wrote it at the very end of the code, even after the join. Here is the proof regarding the 100 rows (there is no filtering by age here, so those 5 rows are loaded again):

Scan csv

number of output rows: 100
number of files read: 1
metadata time: 0 ms
size of files read: 5.7 KiB

FileScan csv

```
[rank#0,name#1,net_worth#2,
 bday#3,age#4,nationality#5]
Batched: false, DataFilters:
[isNotNull(net_worth#2),
 (cast(regex_extract(net_wor
 h#2, ^$(\d+).*$, 1) as int) >
 60), isNotnu..., Format: CSV,
Location:
InMemoryFileIndex[file:/C:/Us
ers/stepa/Desktop/spark_demo/3_Spark_Basics/data/top_100_richest.csv], PartitionFilters:
[], PushedFilters:
[IsNotNull(net_worth)],
ReadSchema:
struct<rank:int,name:string,ne
t_worth:string,bday:string,age:
int,nationality:string>
```

As we can see, there is no filter, and we get all 100 rows. This is **predicate pushdown** in all its glory: the filtered-out rows don't even hit the RAM because the filtering happens instantly.

It doesn't always work out this way, where all filters are pushed as close to the file read as possible. There is a number of transformations that will prevent Spark from pushing filters that far down, simply because those transformations might actually need the very rows the filter would discard.

Another equally important factor is data types. For example, let's say we have a filter on `net_worth`. However, right before this filter, there is a rather complex transformation on that same field, which Spark cannot perform on the fly. Want your filters to be pushed closer to the file read? Perform your filtering using the correct field types. For instance, if it's an `INT` in the file, compare it with an `INT` rather than a string (though Spark will actually manage in this specific case, you might run into nuances with more complex scenarios).

Example: `age` is of type `int`. If you write `col('age') < 40`, it will work and push it down; if you write `col('age') < '40'`, it will also work. But sometimes types cannot be implicitly converted into one another, and that's when things get messy (though Spark will, of course, still try to convert them on the fly in every possible way, all in the name of optimization).

Now let's talk about the second thing you most likely noticed: **PartitionFilters**. It works brilliantly, and I'll explain how it works using `df_parquet` as an example.

Here is what we see in our code: `df_parquet_filtered = df_parquet.filter(col('nationality') == 'Russia')`

We are filtering our Parquet data by the `nationality` field. If you look inside `data_parquet`, you will see a bunch of folders named after specific nationalities. This means that before saving, the data was

split into partitions by the **nationality** field using **partitionBy** (you'll learn why, how, and when to use it later).

If you pay even closer attention, whenever Spark writes something, it always creates additional metadata files indicating what is stored where and how. Because of this, Spark won't even bother checking the rows inside those irrelevant files; it will simply check the metadata first, realize that the rows in those other partitions don't match, and skip them entirely.

The payoff? A massive payoff!!!

Here is what it looks like when filtering is applied:

```
Scan parquet

number of files read: 1
scan time: 609 ms
metadata time: 8 ms
size of files read: 1683.0 B
number of output rows: 9
number of partitions read: 1

FileScan parquet
[rank#96,name#97,net_worth#98,bday#99,age#100,nationality#101] Batched: true,
DataFilters: [], Format:
Parquet, Location:
InMemoryFileIndex[file:/C:/Users/stepa/Desktop/spark_demo/3_Spark_Basics/data_parquet], PartitionFilters:
[isNotNull(nationality#101),
(nationality#101 = Russia)],
PushedFilters: [], ReadSchema:
struct<rank:int,name:string,net_worth:string,bday:string,age:int>
```

Only one partition was detected. And to prove that I'm not lying, here's the output when there's no filter:

Scan parquet

number of files read: 25
scan time total (min, med, max)
281 ms (11 ms, 31 ms, 50 ms)
metadata time: 0 ms
size of files read: 37.8 KiB
number of output rows: 100
number of partitions read: 25

FileScan parquet
[rank#244,name#245,net_worth#246,bday#247,age#248,nationality#249] Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex[file:/C:/Users/stepa/Desktop/spark_demo/3_Spark_Basics/data_parquet], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<rank:int,name:string,net_worth:string,bday:string,age:int>